CO4-AT2-
Industry Problem Solving Task

Name: Ajay Kumar Reddy

Reg.No:192425274

Course: CSA0712 - COMPUTER NETWORKS

Faculty: Dr. S. SALINI

Question 1: Transport Layer Revolution & Connection Management

Traditional web communication relies on TCP, which binds a connection to a fixed combination of network addresses and ports. QUIC changes this by moving transport logic on top of UDP and identifying connections in a completely different way. The three parts below explain the problem, QUIC's fix, and why QUIC still rides on UDP instead of inventing an entirely new transport protocol.

1.1 The Limitations of TCP

The TCP 4-tuple. A standard TCP connection is uniquely identified by four values, together called the 4-tuple:

Component	Example Value (Wi-Fi)	Example Value (4G)
Source IP address	192.168.1.15 (home router NAT)	10.55.2.201 (carrier NAT)
Source port	51000	51000 (kept by OS, but irrelevant)
Destination IP address	203.0.113.10 (server)	203.0.113.10 (server — unchanged)
Destination port	443	443 (unchanged)

The operating system and every middlebox on the path (NAT gateways, firewalls, load balancers) use this 4-tuple purely as a lookup key to route packets to the correct socket and to track the connection's state machine (SYN, ESTABLISHED, etc.). TCP never expected any of these four values to change during the lifetime of a connection.

What happens when the user walks out the door. When a smartphone switches from Wi-Fi to 4G/5G, the device is handed a brand-new IP address by the cellular carrier's network (Wi-Fi and cellular are different physical interfaces, each with its own IP allocated by a different NAT/DHCP domain). Because the source IP address is one of the four fields that define the connection, changing it changes the entire 4-tuple — even though the destination server, the application, and the user's intent have not changed at all.

From the server's perspective, a packet now arrives with a 4-tuple it has never seen before. It does not recognise this as "the same connection continuing on a new path" — it looks like a stray, unsolicited packet. The server has no record of a TCP connection with that new tuple, so it replies with a RST (reset), and the OS-level socket on the phone reports the connection as broken. Any in-flight request (e.g. a video stream, a file download, a video call) is dropped and the application must open a brand-new TCP connection — repeating the full TCP handshake and, for HTTPS, the entire TLS handshake again.

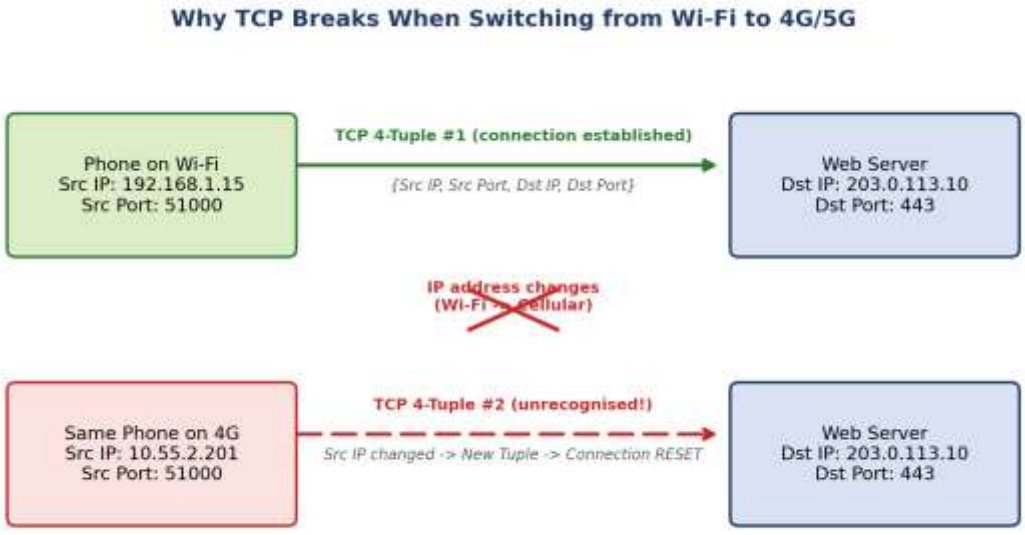


Figure 1.1 — Switching networks changes the source IP, so the TCP 4-tuple no longer matches and the server treats the packet as belonging to an unknown connection.

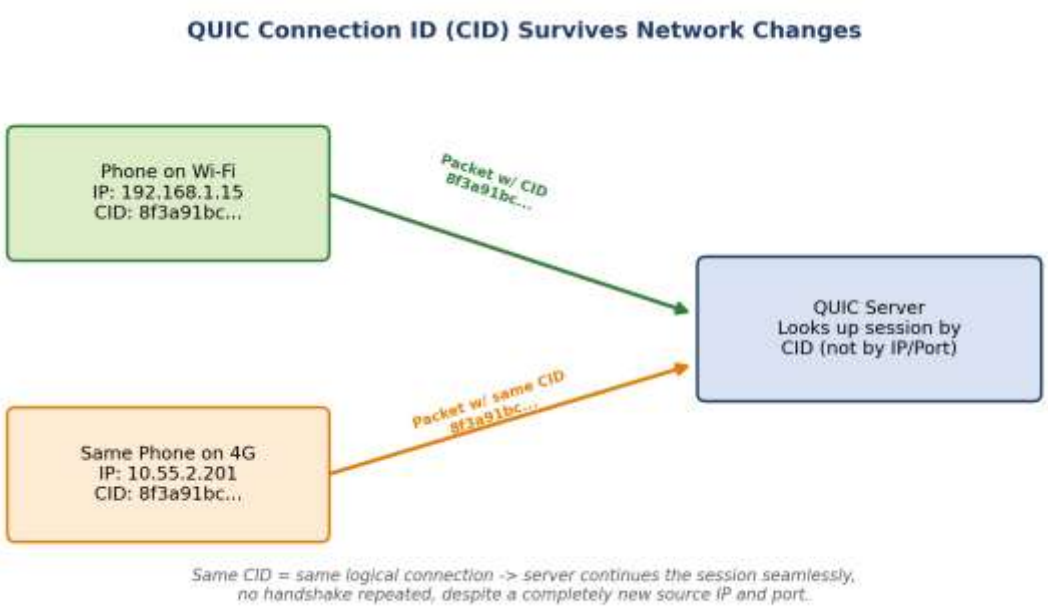
1.2 QUIC Connection ID Mechanism

Decoupling identity from the network path. QUIC solves this by no longer using the IP/port 4-tuple as the primary identifier of a connection. Instead, at the start of a QUIC connection, the client and server each generate and exchange a Connection ID (CID) — a random string of bytes carried in the header of every QUIC packet.

- The CID is chosen independently of the network path (IP address, port, or interface).
- Every QUIC packet — regardless of which Wi-Fi or cellular network it is sent over — carries this same CID in its header.
- The server indexes its connection state (encryption keys, stream buffers, congestion-control state) by CID, not by the 4-tuple.

Recognising a returning user. When the phone switches from Wi-Fi to 4G, its IP address and port change completely, but the QUIC layer keeps reusing the same Connection ID in the packet header. When the first packet from the new 4G address arrives at the server, the server looks inside the QUIC header, extracts the CID, and matches it against its table of active sessions. Finding a match, it simply updates its internal record of "which IP/port this CID is currently reachable at" and continues the session exactly where it left off — no new handshake, no lost data, no visible interruption to the user. This capability is formally called Connection Migration.

A short validation step (a PATH_CHALLENGE / PATH_RESPONSE exchange) is performed so the server does not blindly start sending large amounts of data to a spoofed new address, but this adds negligible delay compared with re-running a full handshake.



The Connection ID stays constant across the Wi-Fi→4G switch, letting the server map both packets to the same ongoing session.

1.3 UDP Overheads

Why not design a new Layer 4 protocol from scratch? In principle, a purpose-built transport protocol could be designed with QUIC's exact feature set baked directly into the IP layer's next protocol number, avoiding any UDP framing overhead. In practice, QUIC's designers rejected this route almost entirely because of ossification of the middle of the Internet.

The Middlebox Problem

The Internet is not just routers forwarding packets — it is full of NAT devices, firewalls, load balancers, and traffic-shaping middleboxes deployed by ISPs, universities, corporate networks, and mobile carriers over the last 30+ years. These devices were built and configured to understand exactly two transport protocols at Layer 4: TCP and UDP. Any packet using a different IP protocol number is very frequently dropped, rate-limited, or actively blocked, because the middlebox has no logic for how to inspect, translate, or forward it safely.

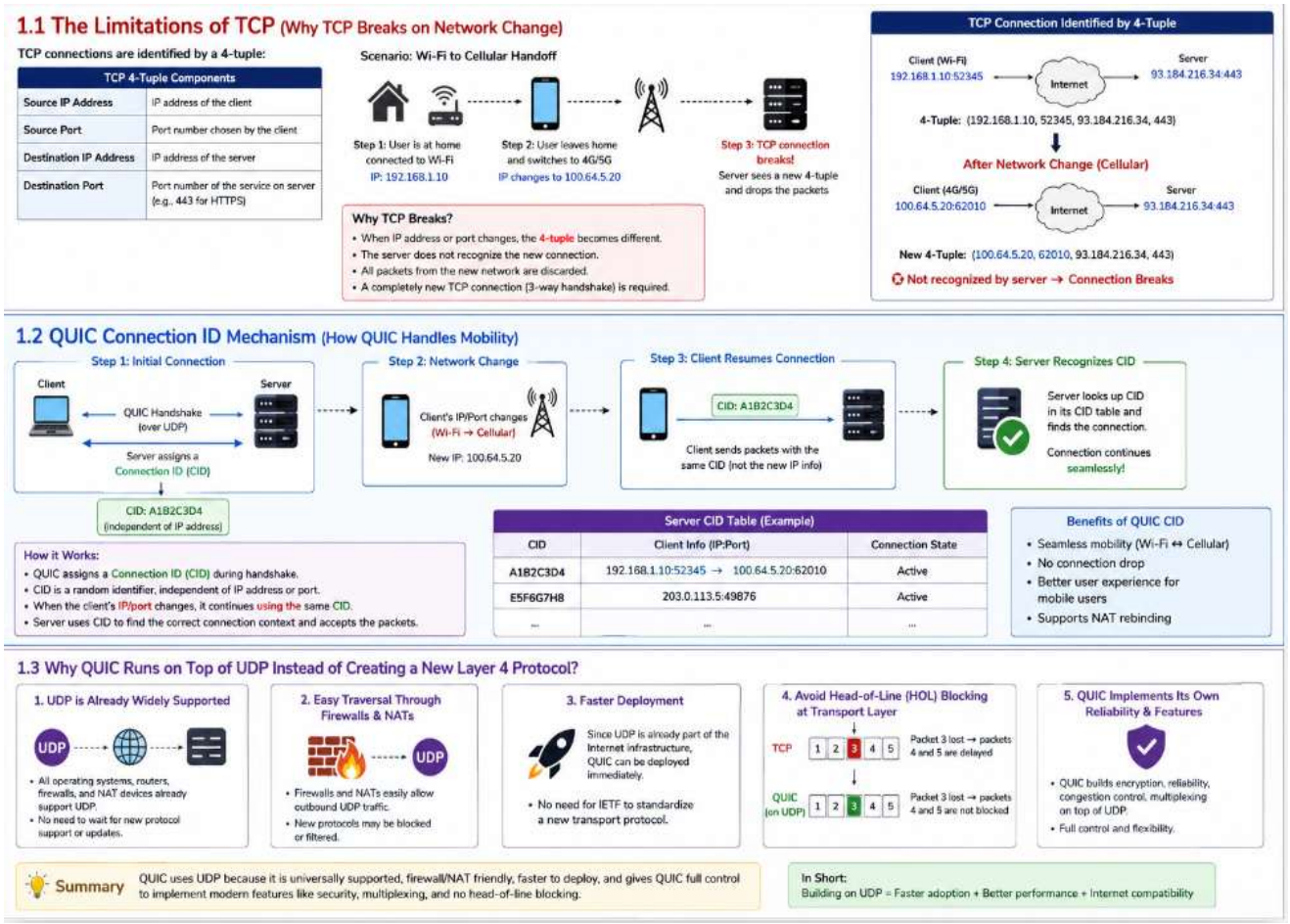
Approach	Deployability on today's Internet	Time to reach users
Brand-new Layer 4 protocol (new IP protocol number)	Very poor — silently dropped by a large fraction of firewalls, NATs and home routers that only recognise TCP/UDP	Years to decades (requires replacing/patching global middlebox fleet)
QUIC built on top of UDP	Excellent — every existing middlebox already forwards UDP datagrams	Immediate — deployable the moment endpoints support it

Why UDP specifically works as the substrate

- UDP is one of the two protocols every NAT/firewall on the planet already knows how to forward and perform NAT translation for.

- UDP itself is deliberately "thin" — it only adds a 8-byte header (source port, destination port, length, checksum) and performs no handshake, ordering, or reliability of its own.
- Because UDP does almost nothing, it leaves QUIC free to implement its own encryption, reliability, ordering, congestion control and multiplexing entirely in user-space, inside the QUIC library itself — without fighting against, or being constrained by, kernel-level TCP behaviour.
- This also means QUIC can evolve and ship new features via ordinary application/library updates, rather than needing an OS kernel update on every device on Earth (which is what a true new Layer 4 protocol, or even changes to TCP itself, would require).

Conclusion. Building on UDP was a pragmatic, deployability-driven decision: it lets QUIC get the traffic-forwarding benefits of an already-universally-supported protocol while implementing all of its genuinely new logic (streams, encryption, connection migration) above it, sidestepping the need to wait for every router, firewall, and NAT box on the Internet to be upgraded to understand a new protocol number.



Question 2: Head-of-Line (HoL) Blocking & Streams

Both HTTP/2 (over TCP) and HTTP/3 (over QUIC) multiplex many logical resource requests — images, stylesheets, scripts — over a single connection. How each protocol behaves when a single packet is lost is where the two diverge sharply.

2.1 The TCP Blockade

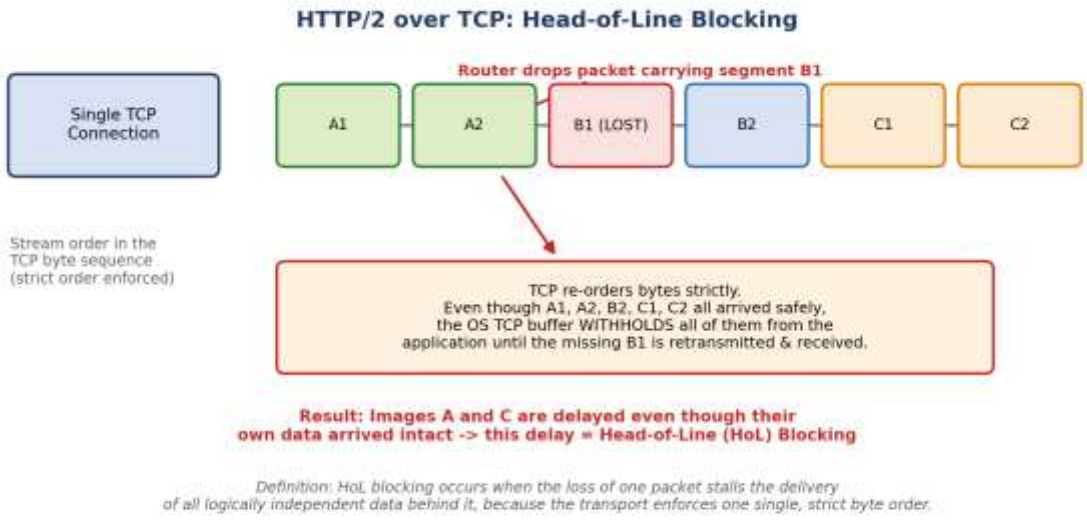
Scenario. A browser opens one HTTP/2 connection over TCP and requests three images — A, B and C — concurrently. HTTP/2 multiplexes their data into interleaved frames, all sent over that single TCP byte stream. Suppose the network drops the packet that happened to be carrying a segment of image B.

TCP guarantees the application only ever sees bytes in the exact order they were sent, with no gaps. This ordering guarantee is enforced at the transport layer, below HTTP/2 entirely — HTTP/2's stream multiplexing is invisible to TCP, which just sees one continuous sequence of bytes to deliver in order.

Because of this, even though the packets carrying data for image A and image C might arrive at the client perfectly intact, the TCP receive buffer in the operating system will not release ANY of that data — including A and C's data that arrived safely — to the browser application. It withholds everything until the lost segment of B is detected (via a duplicate ACK or timeout), retransmitted by the server, and successfully received, so that the byte stream can be reassembled in the correct order.

Definition — Head-of-Line (HoL) Blocking: the delay experienced by data that has already arrived correctly, caused purely because it is queued behind other, unrelated data that is still missing, due to a transport protocol that only knows how to deliver one single, strictly ordered stream of bytes.

In this scenario, the loss of a single packet belonging to image B stalls the delivery of images A and C to the browser as well, even though neither A nor C was actually affected by the packet loss on the wire. The user visibly perceives all three images as "frozen" until the retransmission completes.



TCP enforces one strict, connection-wide byte order, so a single lost segment (B1) blocks the delivery of already-arrived, unrelated data (A and C).

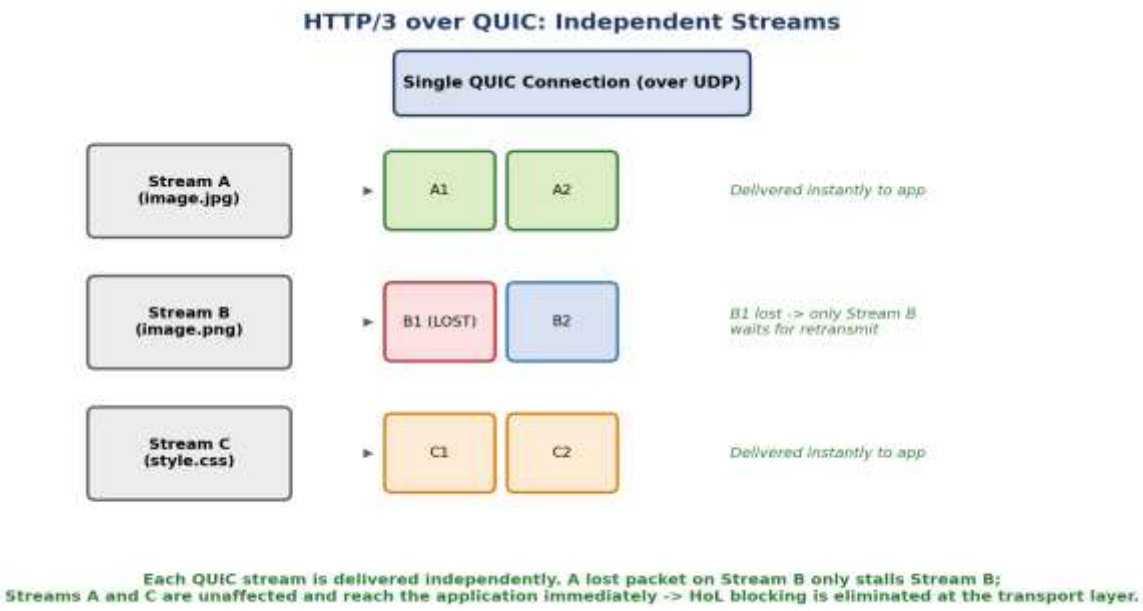
2.2 The QUIC Solution

Streams as first-class, independent objects. QUIC solves this at the transport layer itself, rather than leaving HTTP to work around a transport limitation. Inside a single QUIC connection, each resource (image A, image B, image C) is assigned its own independent stream, and — critically — QUIC's loss detection and retransmission logic operate per-stream, not connection-wide.

Every QUIC packet carries data for one or more streams, tagged with a stream ID and an offset within that stream. When a packet is lost, QUIC's reliability layer only needs to retransmit the specific stream data that was inside that lost packet. It does not need to, and does not, hold back the delivery of data belonging to any other stream.

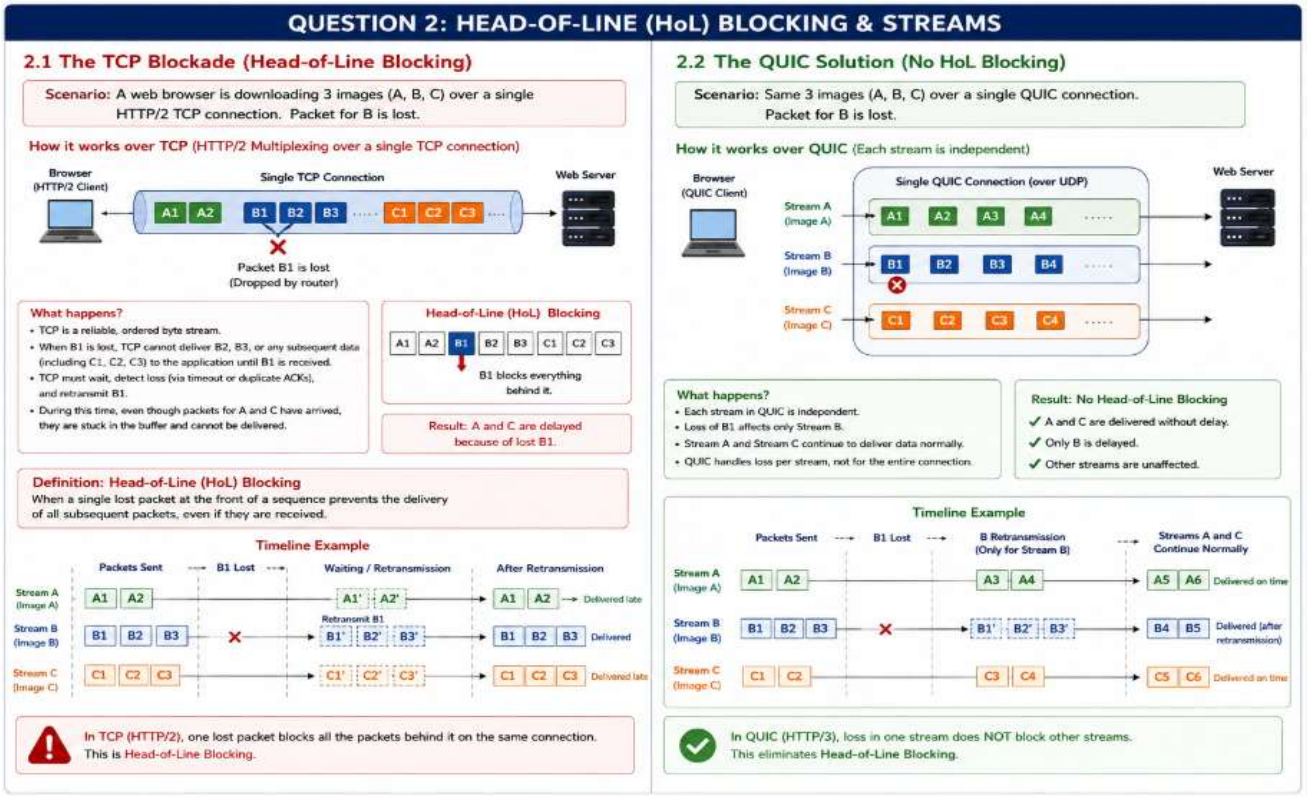
Effect of losing a packet on stream B. If a packet belonging to image B's stream is lost, only image B's stream is paused while QUIC detects the loss and retransmits the missing data. Image A and image C's streams are completely unaffected — because their data was carried in separate packets, tagged with different stream IDs, QUIC already has everything it needs to reassemble and deliver A and C to the browser application immediately, with zero waiting on B.

Aspect	HTTP/2 over TCP	HTTP/3 over QUIC
Ordering guarantee	One strict order for the whole connection	Strict order only within each individual stream
Loss detection & retransmit scope	Whole connection (single byte sequence)	Per-stream only
Effect of 1 lost packet on unrelated streams	All streams stall (HoL blocking)	Only the affected stream stalls
Where multiplexing is implemented	Application layer (HTTP/2 framing) on top of TCP	Transport layer (QUIC) itself



QUIC treats each stream as an independently-ordered sequence. Losing a packet on Stream B only stalls Stream B; Streams A and C are delivered without delay.

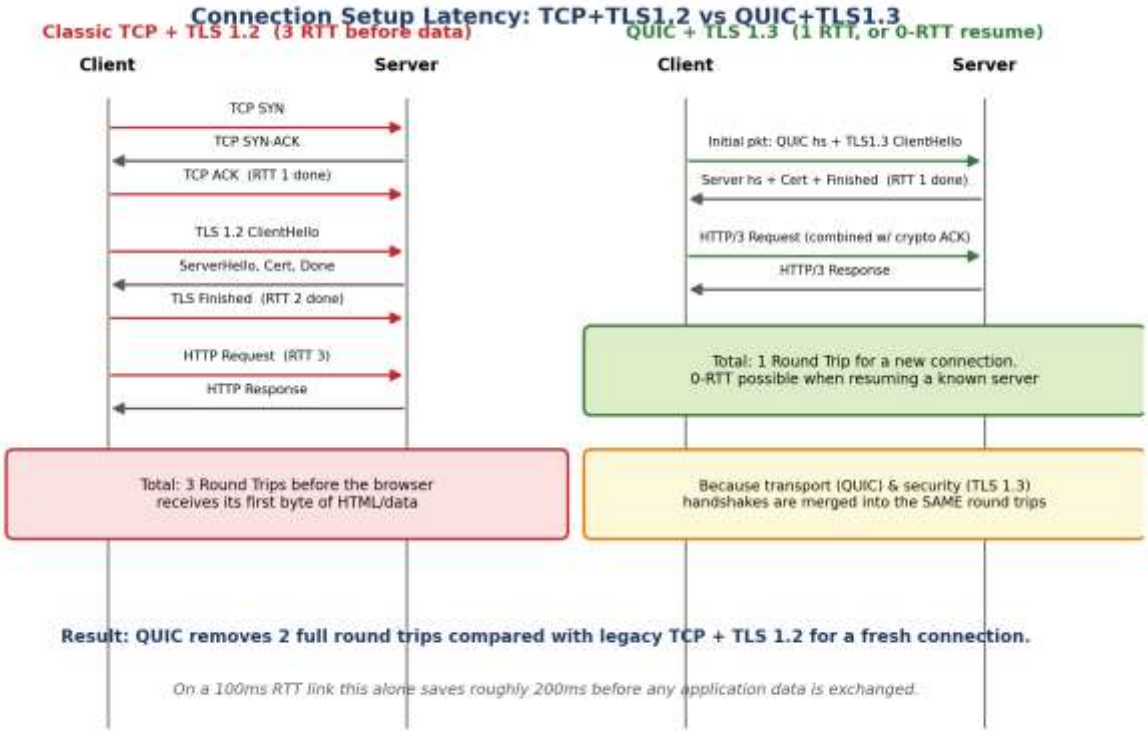
Conclusion. By moving multiplexing down into the transport layer and making loss recovery stream-aware, QUIC eliminates transport-level Head-of-Line blocking entirely — a structural improvement that HTTP/2 over TCP cannot achieve no matter how it is tuned, because the limitation lives in TCP itself.



Question 3: Handshake Mechanics & RTT

3.1 Step-by-Step Latency Breakdown

Establishing a brand-new, secure (HTTPS) connection historically required stacking two separate handshakes on top of each other: the TCP three-way handshake, followed only afterwards by the TLS handshake. QUIC instead merges the transport and cryptographic handshakes into the same round trips.



Sequence diagram comparing round trips: legacy TCP + TLS 1.2 (3 RTT) versus QUIC + TLS 1.3 (1 RTT for a new connection).

Classic TCP + TLS 1.2 — round trip by round trip

RTT #	Messages exchanged	Purpose
RTT 1	SYN -> / <- SYN-ACK / ACK ->	TCP three-way handshake: establishes the transport-layer connection only. No security, no data yet.
RTT 2	ClientHello -> / <- ServerHello, Certificate, ServerHelloDone / Finished ->	TLS 1.2 handshake: negotiates cipher suite, exchanges/validates certificates, derives session keys.
RTT 3	HTTP Request -> / <- HTTP Response	First actual application data (e.g. the HTML page) can finally be requested and returned.

QUIC + TLS 1.3 — round trip by round trip

RTT #	Messages exchanged	Purpose
RTT 1	Initial packet (QUIC transport params + TLS 1.3 ClientHello) -> / <- Server's transport params + Certificate + Finished	Transport AND cryptographic handshakes happen together, inside the same exchange.
Same RTT	HTTP/3 request can be sent immediately after, piggybacked with the handshake-completion flight	Application data starts flowing after just 1 RTT.
0-RTT (repeat visits only)	Client sends encrypted application data immediately, using resumption keys from a previous session	Skips the round trip entirely when reconnecting to a previously-visited server.

3.2 Handshake Optimisation

Root cause of the speed-up: merged layers. In the legacy stack, TCP and TLS are two independent protocols, implemented and standardised separately, and neither is aware of the other. As a strict architectural rule, TLS cannot begin its own handshake until the TCP three-way handshake has fully completed and an ordered, reliable byte stream exists for it to run over. This layering forces the two handshakes to happen sequentially, one after another, costing 2 full round trips before any encrypted TLS data can move, and a 3rd round trip before application data is exchanged.

QUIC was designed from the outset with TLS 1.3 built directly into its own handshake — the QUIC transport parameters (equivalent to what TCP's SYN negotiates) and the TLS 1.3 ClientHello/ServerHello (which derive the encryption keys) are carried inside the very same packets, in the very same round trip. There is no artificial requirement to wait for one layer to finish before starting the other, because QUIC does not treat transport setup and security setup as separate protocols running on top of each other — it treats them as one integrated handshake.

TLS 1.3 itself is also independently faster than TLS 1.2: it removed an entire round trip from the older TLS 1.2 negotiation by cutting down the cipher-suite negotiation and combining key-exchange messages into a single flight. QUIC benefits from this TLS 1.3 efficiency on top of merging the layers.

Net result. For a brand-new connection, this reduces setup time from 3 RTTs (TCP + TLS 1.2) down to just 1 RTT (QUIC + TLS 1.3) — and QUIC's 0-RTT resumption feature (using cached session keys from an earlier visit) can remove that final round trip entirely for a returning user, letting encrypted application data be sent on the very first flight. On a mobile connection with, for example, a 100 ms round-trip time, moving from 3 RTT to 1 RTT alone saves roughly 200 ms of pure waiting time before the page can even start loading — a difference that is very noticeable to end users, especially on high-latency mobile networks.

Stack	Round trips before app data	Approx. time saved (@100ms RTT)
TCP + TLS 1.2	3 RTT	Baseline
QUIC + TLS 1.3 (new connection)	1 RTT	~200 ms saved
QUIC + TLS 1.3 (0-RTT resumption)	0 RTT	~300 ms saved

In summary, QUIC is faster to start not because any single step is individually quicker, but because it restructures the handshake so that transport setup and encryption setup are no longer forced to happen as two separate, sequential negotiations.

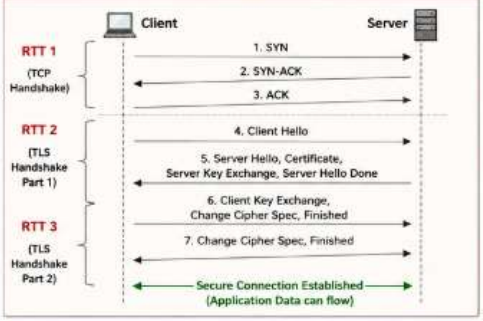
QUESTION 3: HANDSHAKE MECHANICS & RTT [15 MARKS]

3.1 STEP-BY-STEP LATENCY BREAKDOWN (8 MARKS)

Below shows the network round-trips (RTTs) required to establish a secure web connection using **Classic TCP + TLS 1.2** vs. **QUIC + TLS 1.3**.

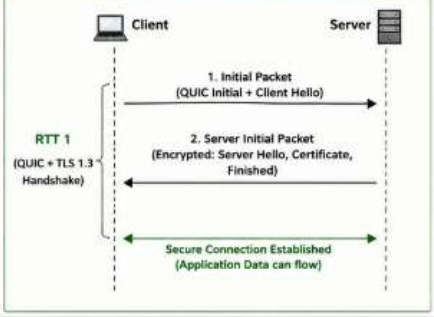
A. CLASSIC TCP + TLS 1.2 HANDSHAKE

Total: 3 RTTs (before any application data)



B. QUIC + TLS 1.3 HANDSHAKE

Total: 1 RTT (often 0 RTT resumption)



COMPARISON SUMMARY

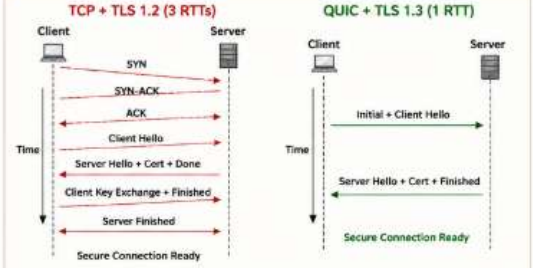
Feature	TCP + TLS 1.2	QUIC + TLS 1.3
Transport Layer	TCP	UDP (QUIC)
TLS Version	TLS 1.2	TLS 1.3
RTTs for Handshake	3 RTTs	1 RTT (0 RTT resumption)
Connection Start	Slower	Faster
Encryption Start	After 3 RTTs	After 1 RTT
Packet Loss Impact	High (TCP HOL blocking)	Low (QUIC handles loss)
Multiplexing	Limited	Built-in

3.2 HANDSHAKE OPTIMIZATION (7 MARKS)

WHY QUIC IS SIGNIFICANTLY FASTER?

1. **Fewer Round Trips:** TCP + TLS 1.2 requires 3 RTTs (1 for TCP handshake + 2 for TLS handshake). QUIC + TLS 1.3 completes in just 1 RTT (often 0 RTT with session resumption).
2. **TLS 1.3 Efficiency:** TLS 1.3 reduces handshake messages by combining steps and removing unnecessary exchanges (e.g., no separate change cipher spec).
3. **Integrated Handshake:** QUIC integrates transport and TLS handshake in the first packet, eliminating the need for a separate TCP handshake.
4. **0-RTT Resumption:** QUIC with TLS 1.3 supports 0-RTT resumption, allowing clients to send application data immediately using cached session tickets.
5. **Better Loss Handling:** QUIC runs over UDP and handles packet loss at the application layer, avoiding TCP's head-of-line (HOL) blocking, so lost packets do not delay the entire connection.
6. **Built-in Multiplexing:** Multiple streams can be sent over a single QUIC connection without blocking, improving performance for modern web apps.
7. **Faster Connection Start = Better User Experience:** Websites load quicker, especially on high-latency or lossy networks (mobile, Wi-Fi, satellite).

VISUAL SUMMARY



KEY TAKEAWAY

QUIC + TLS 1.3 significantly reduces connection setup time by minimizing round trips, integrating handshakes, and improving loss handling, making it the preferred choice for modern internet applications.

Question 4: Handshake Simplification & Speed

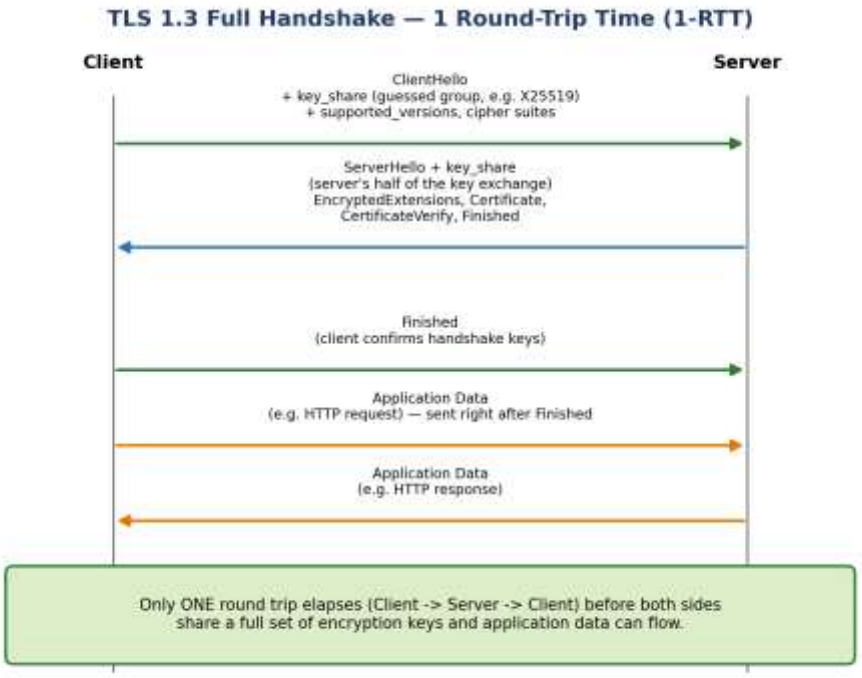
TLS 1.3 was a ground-up redesign of the TLS handshake, aimed squarely at cutting both latency and attack surface. The three parts below cover the full 1-RTT handshake, the even faster 0-RTT resumption mode, and the security cost that 0-RTT carries.

4.1 The 1-RTT Handshake

The old problem TLS 1.3 removed. In TLS 1.2, the client would first send a ClientHello simply listing the cipher suites and key-exchange groups it supported, and only after hearing back from the server which one was actually chosen could the real key-exchange values be generated and sent. That negotiate-then-exchange sequence cost an extra round trip.

The crucial guess. TLS 1.3 collapses this into a single round trip by having the client guess, in its very first ClientHello, which key-exchange group the server is most likely to support (in practice almost always a well-known, near-universal choice such as X25519 or secp256r1) and immediately compute and attach its Diffie-Hellman key_share for that group — before the server has confirmed it will accept it.

Because most modern servers do in fact support that commonly-guessed group, the server can, in the overwhelming majority of cases, immediately reply in a single flight with its own key_share, certificate, CertificateVerify, and Finished message — everything needed to complete the cryptographic handshake in one go.



TLS 1.3's guessed key_share lets the server reply with its full handshake flight immediately, completing the handshake in exactly 1-RTT.

What happens on a wrong guess. If the client's guessed group is not one the server supports, the server sends a HelloRetryRequest telling the client which group to use instead, and the client retries — this costs an extra round trip, but only in this uncommon fallback case. For the vast majority of real-world connections, the guess succeeds

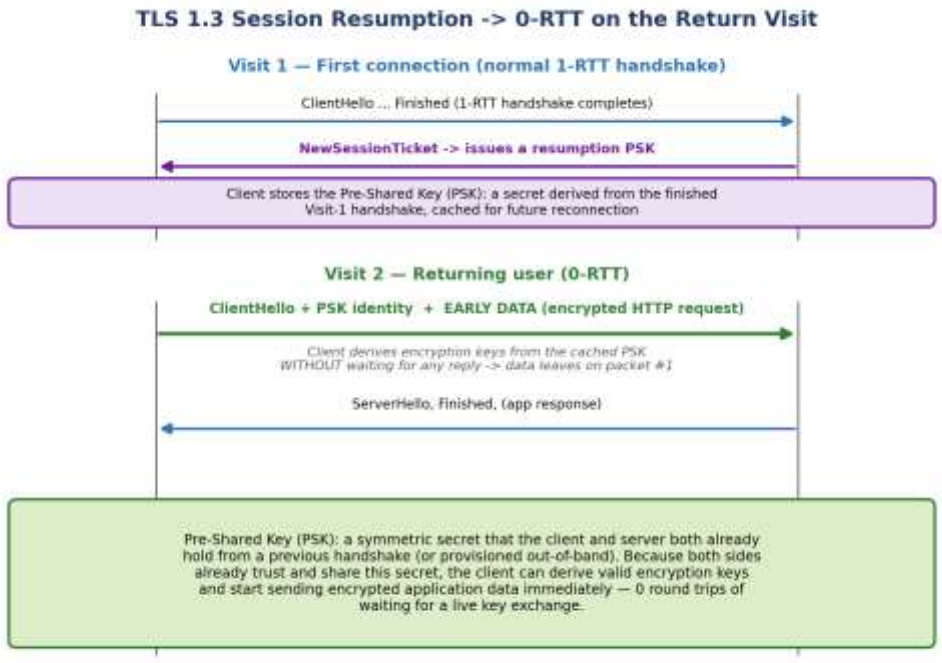
and the full handshake — key agreement, server authentication, and readiness to exchange application data — completes in exactly 1 Round-Trip Time.

4.2 Zero Round-Trip Time (0-RTT) Mode

Session resumption in two visits. 0-RTT only works for a returning user who has connected to that exact server before. On that first ("Visit 1") connection, after the normal 1-RTT handshake completes, the server sends the client a NewSessionTicket message. This ticket carries the material needed to derive a Pre-Shared Key (PSK) for next time, which the client caches locally.

What a Pre-Shared Key (PSK) is. A PSK is a symmetric secret that both the client and the server already possess before any new connection begins — in this case, derived from the successfully completed Visit-1 handshake (a PSK can also be provisioned out-of-band, but session-ticket resumption is the common web case). Because both parties already trust and hold this shared secret, neither side needs to perform a fresh, live key exchange to agree on new encryption keys — the PSK itself can be used directly to derive them.

Sending data in the very first packet. On "Visit 2", the client's first packet to the server is a ClientHello that references the cached PSK (via a PSK identity) and, in the very same flight, appends "early data": an already-encrypted payload (for example, an HTTP GET request), encrypted using keys the client derived on its own from the cached PSK. There is no need to wait for any reply from the server before sending this — hence "zero" round-trip time. The server, holding the same PSK, can decrypt and act on this early data immediately upon receipt, before its own handshake reply has even been generated.



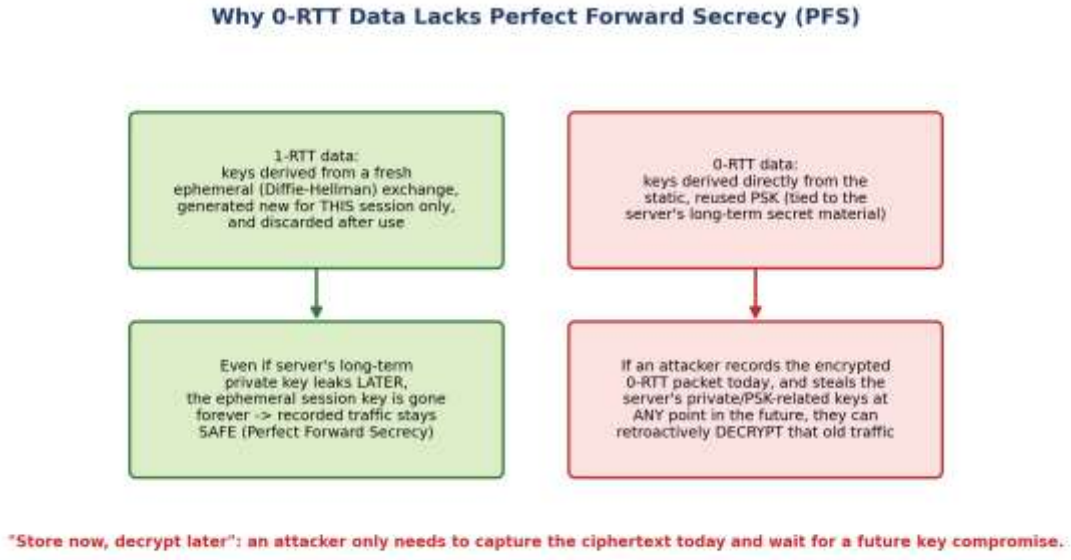
A PSK issued at the end of Visit 1 lets the client encrypt and send application data in the very first packet of Visit 2, with zero round trips of waiting.

4.3 Security Trade-off

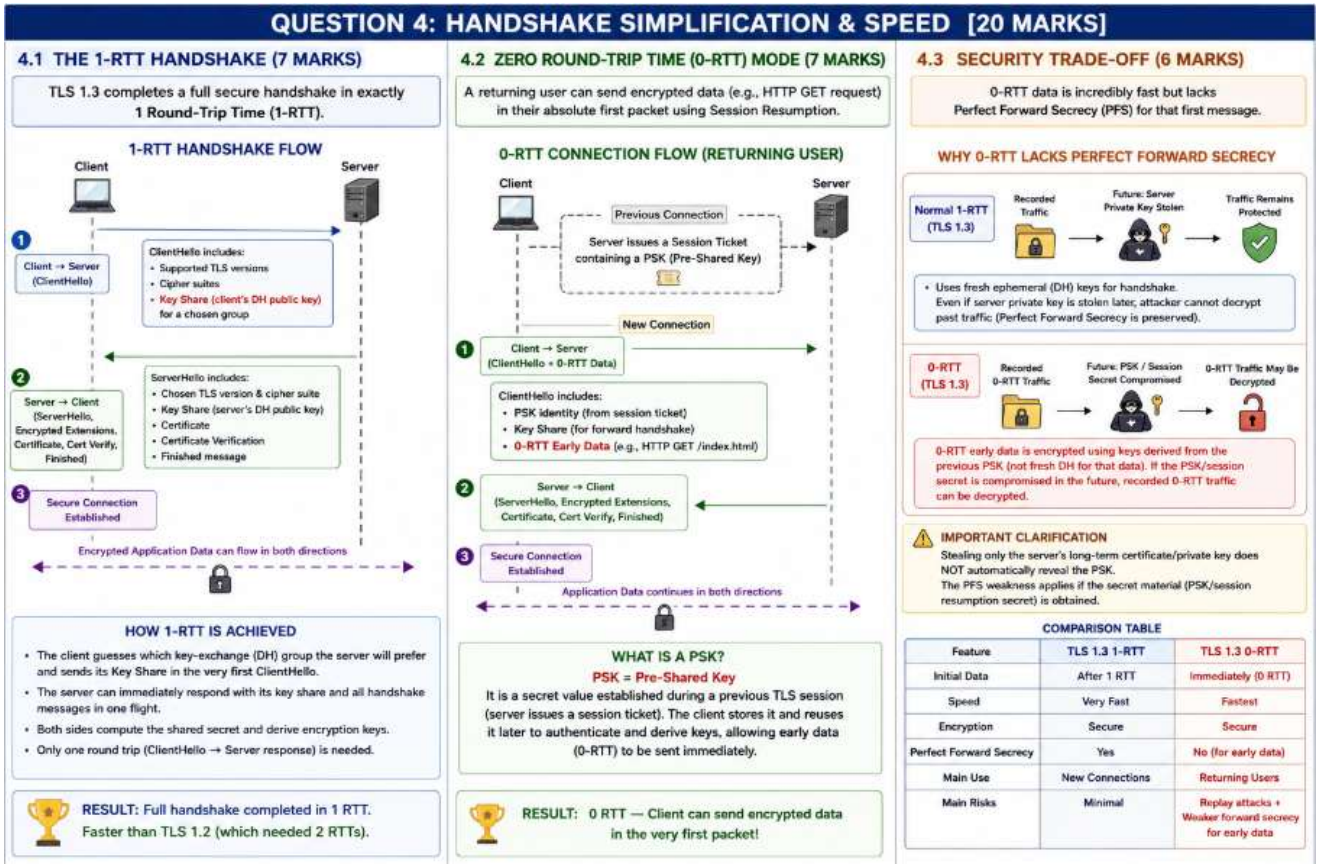
What Perfect Forward Secrecy normally guarantees. In a full 1-RTT handshake, the encryption keys are derived from a fresh, ephemeral Diffie-Hellman exchange generated new for that single session and then thrown away. This means that even if the server's long-term private key is stolen at some point in the future, that theft is useless for decrypting old, already-recorded 1-RTT traffic — the ephemeral session keys used back then no longer exist anywhere and cannot be reconstructed from the long-term key alone. This property is Perfect Forward Secrecy (PFS).

Why 0-RTT data does not get this protection. The very first flight of 0-RTT data is encrypted using keys derived directly from the static, reused PSK, rather than from a brand-new ephemeral exchange (there simply hasn't been time for a fresh live exchange yet — that is the whole point of skipping the round trip). This means the keys protecting that first 0-RTT message are tied back to longer-lived secret material.

The consequence if the server's private keys are stolen later. If an attacker has been passively recording a victim's encrypted 0-RTT traffic — a "store now, decrypt later" strategy — and at some future date manages to steal the server's private key (or the PSK-related secret material), the attacker can go back and use that stolen key to derive the same keys that protected the old recorded 0-RTT packets, and decrypt them retroactively. Traffic that felt safely encrypted at the time it was sent can become readable in hindsight, purely because of a compromise that happens much later. This is exactly the guarantee that Perfect Forward Secrecy is designed to prevent, and exactly the guarantee that only the very first 0-RTT flight of data does not receive (all the subsequent 1-RTT data in that same connection, once the live handshake completes, is protected by fresh ephemeral keys as normal).



1-RTT keys are ephemeral and self-destruct after the session; 0-RTT's very first flight is tied to the reusable PSK, so a future key compromise can expose it retroactively.


QUESTION 4: HANDSHAKE SIMPLIFICATION & SPEED [20 MARKS]
4.1 THE 1-RTT HANDSHAKE (7 MARKS)
TLS 1.3 completes a full secure handshake in exactly 1 Round-Trip Time (1-RTT).
1-RTT HANDSHAKE FLOW
Client
Server
1
Client → Server (ClientHello)
ClientHello includes:
• Supported TLS versions
• Cipher suites
• Key Share (client's DH public key) for a chosen group
2
Server → Client (ServerHello, Encrypted Extensions, Certificate, Cert Verify, Finished)
ServerHello includes:
• Chosen TLS version & cipher suite
• Key Share (server's DH public key)
• Certificate
• Certificate Verification
• Finished message
3
Secure Connection Established
Encrypted Application Data can flow in both directions
HOW 1-RTT IS ACHIEVED
• The client guesses which key-exchange (DH) group the server will prefer and sends its Key Share in the very first ClientHello.
• The server can immediately respond with its key share and all handshake messages in one flight.
• Both sides compute the shared secret and derive encryption keys.
• Only one round trip (ClientHello → Server response) is needed.
RESULT: Full handshake completed in 1 RTT.
Faster than TLS 1.2 (which needed 2 RTTs).
4.2 ZERO ROUND-TRIP TIME (0-RTT) MODE (7 MARKS)
A returning user can send encrypted data (e.g., HTTP GET request) in their absolute first packet using Session Resumption.
0-RTT CONNECTION FLOW (RETURNING USER)
Client
Server
Previous Connection
Server issues a Session Ticket containing a PSK (Pre-Shared Key)
New Connection
1
Client → Server (ClientHello + 0-RTT Data)
ClientHello includes:
• PSK identity (from session ticket)
• Key Share (for forward handshake)
• 0-RTT Early Data (e.g., HTTP GET /index.html)
2
Server → Client (ServerHello, Encrypted Extensions, Certificate, Cert Verify, Finished)
3
Secure Connection Established
Application Data continues in both directions
WHAT IS A PSK?
PSK = Pre-Shared Key
It is a secret value established during a previous TLS session (server issues a session ticket). The client stores it and reuses it later to authenticate and derive keys, allowing early data (0-RTT) to be sent immediately.
RESULT: 0 RTT — Client can send encrypted data in the very first packet!
4.3 SECURITY TRADE-OFF (6 MARKS)
0-RTT data is incredibly fast but lacks Perfect Forward Secrecy (PFS) for that first message.
WHY 0-RTT LACKS PERFECT FORWARD SECRECY
Normal 1-RTT (TLS 1.3)
Recorded Traffic
Future: Server Private Key Stolen
Traffic Remains Protected
• Uses fresh ephemeral (DH) keys for handshake.
Even if server private key is stolen later, attacker cannot decrypt past traffic (Perfect Forward Secrecy is preserved).
0-RTT (TLS 1.3)
Recorded 0-RTT Traffic
Future: PSK / Session Secret Compromised
0-RTT Traffic May Be Decrypted
0-RTT early data is encrypted using keys derived from the previous PSK (not fresh DH for that data). If the PSK/session secret is compromised in the future, recorded 0-RTT traffic can be decrypted.
IMPORTANT CLARIFICATION
Stealing only the server's long-term certificate/private key does NOT automatically reveal the PSK.
The PFS weakness applies if the secret material (PSK/session resumption secret) is obtained.
COMPARISON TABLE
Feature | TLS 1.3 1-RTT | TLS 1.3 0-RTT
Initial Data | After 1 RTT | Immediately (0 RTT)
Speed | Very Fast | Fastest
Encryption | Secure | Secure
Perfect Forward Secrecy | Yes | No (for early data)
Main Use | New Connections | Returning Users
Main Risks | Minimal | Replay attacks + Weaker forward secrecy for early data

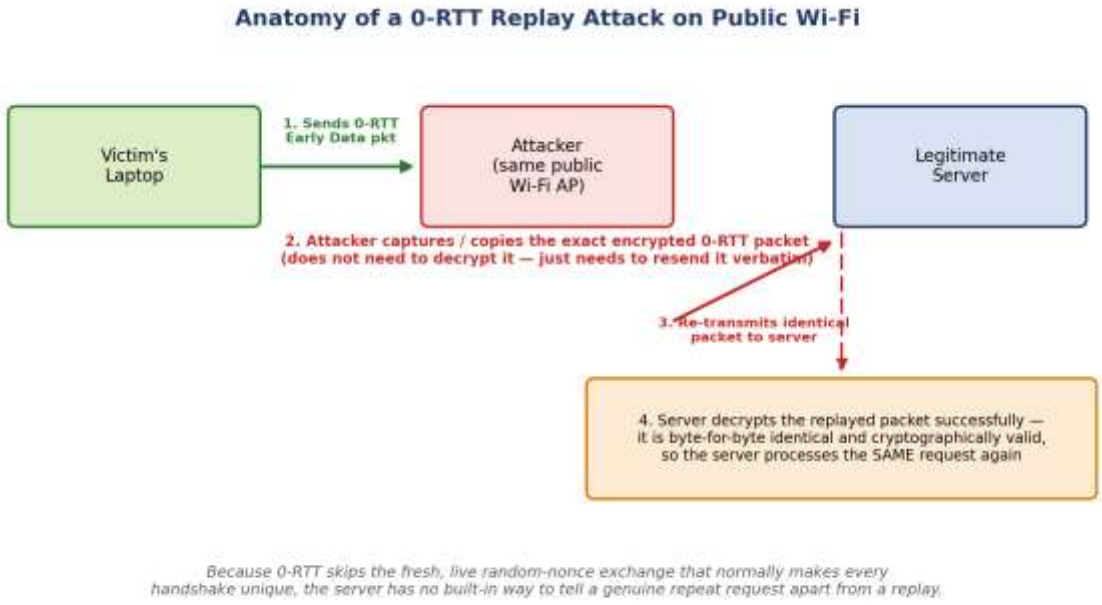
Question 5: The Replay Attack Threat

0-RTT's speed comes directly from skipping the fresh, live exchange of random values that a normal handshake performs. That same missing freshness is precisely what opens the door to Replay Attacks.

5.1 Anatomy of a Replay Attack

A public university Wi-Fi network is a classic on-path attacker setting: because the network is open or uses a shared/weak passphrase, an attacker connected to the same access point can position themselves to observe and capture other users' traffic passing over that local segment (or via a rogue access point / ARP-spoofing position acting as a man-in-the-middle).

Step	What happens
1. Capture	A victim on the same Wi-Fi sends a 0-RTT early-data packet (e.g. an encrypted HTTP request) to a real server. The attacker, sitting on the same network segment, captures a verbatim copy of this encrypted packet.
2. No decryption needed	The attacker does NOT need to break the encryption or know the PSK. TLS 1.3's 0-RTT data is not tied to any fresh, per-attempt randomness that would make a resend detectable as different — the packet is simply valid ciphertext that the server already knows how to decrypt.
3. Replay	The attacker resends that exact same captured packet to the server again — once, or many times — either shortly after or even after some delay.
4. Server re-processes it	Because the packet is byte-for-byte identical and passes all of TLS 1.3's normal cryptographic checks (it is a validly-encrypted message under a PSK the server recognises), the server has no straightforward way, at the TLS layer alone, to tell this apart from the victim genuinely sending the same request twice — and processes it again.



On shared public Wi-Fi, an attacker only needs to capture and resend the encrypted 0-RTT packet verbatim; no decryption is required to make the server act on it again.

This is possible specifically because 0-RTT skips the live ClientHello/ServerHello random-nonce exchange that, in a full handshake, makes every single connection attempt cryptographically unique and therefore un-replayable — a 1-RTT (or later) exchange would never accept a byte-for-byte repeat of an old handshake.

5.2 Safe vs. Unsafe Actions

Why GET /news is harmless to replay. An HTTP GET request is defined to be idempotent and "safe" in the HTTP specification sense: it is only supposed to retrieve data and must not change any state on the server. Whether the server processes that GET request once or fifty times, the end result for the server's data is identical — the news article gets read/returned again, nothing is created, charged, or modified. Replaying it wastes a small amount of bandwidth and server effort, but causes no real damage.

Why POST /api/buy-item is highly dangerous to replay. A POST request like /api/buy-item is explicitly meant to change server-side state — in this case, to trigger a purchase (charge a payment method, decrement inventory, create an order record). It is not idempotent: processing it once means "buy 1 item", and if an attacker replays that exact same captured packet, the server may honestly believe it received a second, independent purchase request and process it again — meaning the victim could be charged multiple times, or have multiple duplicate orders created, for a single action they only performed once.

Request type	Effect of replaying it multiple times	Danger level
GET /news (read-only, idempotent)	Same article returned again — no state change	Harmless
POST /api/buy-item (state-changing, non-idempotent)	Purchase, payment, or order may be duplicated for every replay	Highly dangerous

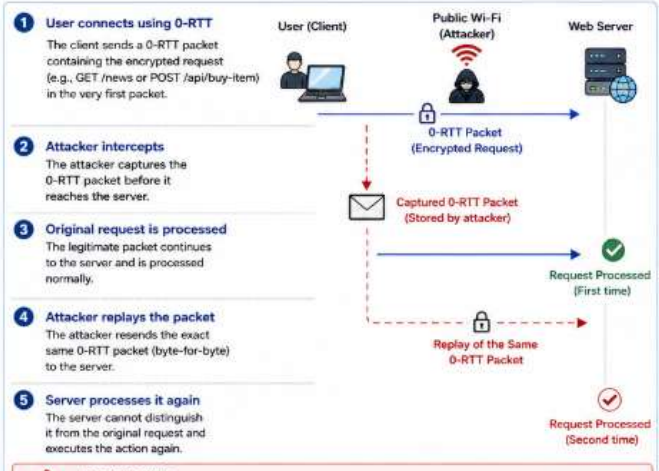
How developers should protect their APIs. The TLS 1.3 specification itself explicitly warns that 0-RTT data must only be used for requests that are safe to receive more than once, and recommends application-layer defences on top, since TLS cannot fully solve this by itself:

- Never allow 0-RTT early data to trigger any non-idempotent / state-changing operation (payments, account changes, item purchases) — servers can and should reject early data for such endpoints and force those requests to wait for the full handshake to complete.
- Use idempotency keys / anti-replay tokens: the client attaches a unique, single-use token to each state-changing request, and the server keeps a short-lived record of tokens it has already processed, rejecting any request that reuses one it has seen before.
- Use nonces or timestamps with a tight validity window combined with server-side tracking, so that even a captured, valid request cannot be usefully resent much later.
- Restrict 0-RTT support at the server/TLS-library configuration level to only the specific routes or endpoints known to be safe (read-only), rather than enabling it connection-wide.

QUESTION 5: THE REPLAY ATTACK THREAT [15 MARKS]

5.1 ANATOMY OF A REPLAY ATTACK (8 MARKS)

An attacker on a public Wi-Fi can capture a user's 0-RTT packet and replay it to the server to repeat the same action.



Key Takeaway
Because 0-RTT data is accepted without a fresh exchange of random values, an attacker can capture and replay it, causing the server to repeat the action.

Replay Attack Summary

1	Victim sends 0-RTT packet with request.
2	Attacker captures the encrypted packet on the network.
3	Server processes the original request.
4	Attacker resends (replays) the identical packet.
5	Server processes it again, thinking it is a valid new request.

Where can this happen?

- Public Wi-Fi in universities, cafes, airports, libraries
- Any open or shared network where attacker can sniff traffic
- Attacker needs only packet capture and replay tools

5.2 SAFE VS. UNSAFE ACTIONS (7 MARKS)

Some requests are safe to replay, while others can cause serious harm. Developers must design APIs to prevent attacks.

SAFE TO REPLAY (Generally Read-Only)

Example: GET /news, GET /profile?id=123

- These requests only retrieve data.
- Replaying them returns the same response.
- No state change on the server.
- Harmless to replay.

Impact: Low

UNSAFE TO REPLAY (State-Changing)

Example: POST /api/buy-item, POST /api/transfer-money

- These requests change server state.
- Replaying can duplicate actions (e.g., multiple purchases or transfers).
- Can cause financial loss, data corruption, or account abuse.

Impact: High (Very Dangerous)

How Should Developers Protect Their APIs?

Disable 0-RTT for Unsafe Endpoints
Do not allow 0-RTT early data for state-changing requests (e.g., POST, PUT, DELETE). Allow only for safe GET requests.

Use Anti-Replay Tokens (Nonces)
Require a unique, single-use token or nonce with each request. The server stores and rejects any reused token.

Timestamps + Expiry Window
Include a timestamp in the request and accept only if it is within a short, valid time window.

Bind to Session / Client
Bind tokens to the user session or client so they cannot be reused from another context.

Idempotency Keys for POST Requests
Use idempotency keys so the server can detect and ignore duplicate requests.

Server-Side Rate Limiting & Monitoring
Detect unusual duplicate requests and block or challenge suspicious activity.

Bottom Line

- Replay attacks exploit the speed of 0-RTT by reusing captured packets.
- Read-only requests are usually safe; state-changing requests are risky.
- Developers must disable 0-RTT for unsafe actions and use anti-replay protections.

Question 6: Cryptographic Cleaning & Protocol Downgrades

6.1 Deprecation of Weak Ciphers

TLS 1.2 supported a large menu of optional cryptographic algorithms, including ones that were already known to be weak by the time TLS 1.2 was in wide use — such as Static RSA key exchange and old hash functions like MD5. TLS 1.3 did not just discourage these; it removed them from the protocol entirely, along with other legacy mechanisms like RC4, CBC-mode ciphers with padding-oracle weaknesses, and compression.

Why simply leaving them as "optional, but not recommended" is not good enough. Making an algorithm merely optional still leaves it reachable and negotiable by anyone on the wire, which creates several concrete, exploitable problems:

- Downgrade / negotiation attacks: if an old, weak option is still technically present in the protocol's cipher-suite list, an on-path attacker can attempt to manipulate the handshake negotiation so the two endpoints "agree" to use the weak option instead of a strong one — even if both sides would have preferred the strong one by default.
- Server / client misconfiguration: real-world deployments are frequently configured by non-experts, or left on old default settings for years. If a weak cipher is technically still legal within the standard, some fraction of servers on the Internet will end up enabling it — whether by accident, laziness, or backward-compatibility pressure — and become exploitable.
- A larger, more error-prone codebase: every additional optional algorithm is more code that has to be implemented, tested, and kept free of bugs in every TLS library on Earth. Static RSA key exchange in particular does not provide forward secrecy at all (a single stolen private key can decrypt all past traffic protected by it), and keeping it "available" preserves that systemic risk indefinitely.
- MD5 and similar broken hash functions were, by the time TLS 1.3 was designed, already practically breakable for producing hash collisions — retaining them as a legal choice keeps a known, demonstrated attack technique viable against any implementation or configuration that still permits them.

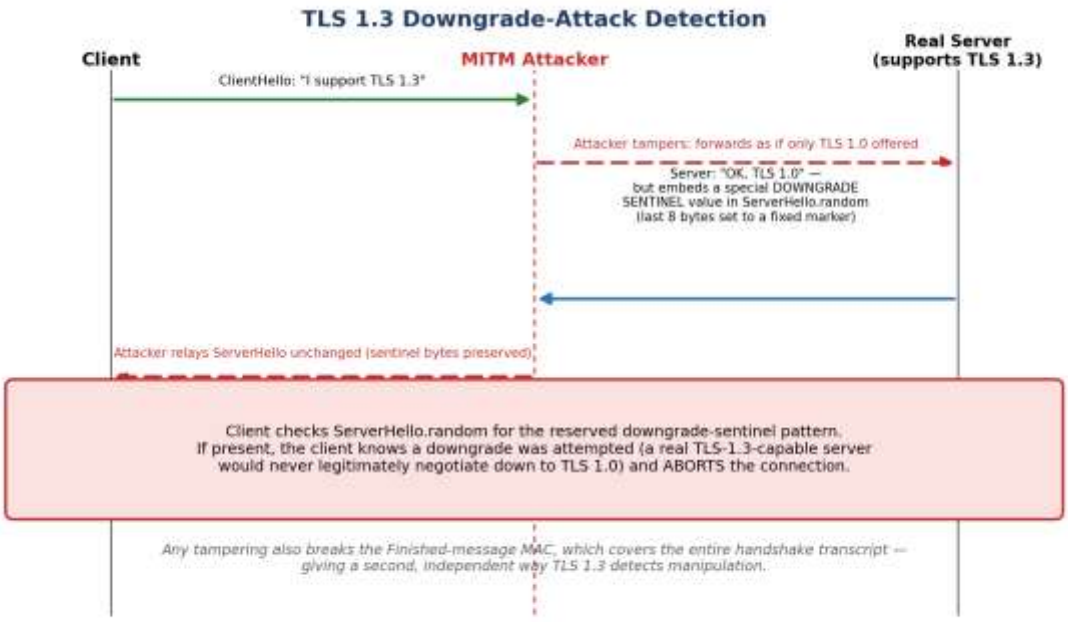
The security benefit of outright removal. By deleting these algorithms from the specification entirely rather than just marking them as discouraged, TLS 1.3 makes it structurally impossible for a compliant implementation to negotiate them at all — there is no configuration mistake, downgrade trick, or legacy fallback that can bring a known-weak algorithm back into play. This shrinks the attack surface for every single TLS 1.3 deployment simultaneously and permanently, rather than relying on every individual administrator, years into the future, to remember to keep the weak options switched off.

6.2 Preventing Downgrade Attacks

The attack scenario. A Man-in-the-Middle attacker sits between a TLS-1.3-capable client and a TLS-1.3-capable server. The attacker intercepts the client's ClientHello (which lists TLS 1.3 as a supported version) and modifies it — or intercepts and modifies the subsequent negotiation — so that the server is tricked into believing the client only supports an older, weaker version such as TLS 1.0, hoping both sides will "agree" to fall back to that weaker, more attackable protocol.

The downgrade-sentinel defence. To defend against exactly this, whenever a TLS-1.3-capable server ends up negotiating a connection down to TLS 1.2 or earlier, it is required to set specific, publicly-known fixed values into the last 8 bytes of the random field inside its ServerHello message — a special reserved bit pattern (a different

fixed value is defined for a downgrade to TLS 1.2 versus TLS 1.1/1.0) that a genuine, unmanipulated negotiation to that lower version would never naturally produce.



A TLS-1.3-capable server marks any negotiated downgrade with a reserved sentinel value in ServerHello.random, which the client checks for and rejects if a downgrade was not genuinely necessary.

Critically, this sentinel is inserted by the server itself, based on what version it actually supports and is really willing to offer — the attacker cannot forge it, because doing so would require knowing/predicting values that are defined as part of the honest protocol behaviour of a capable server, not something the attacker (who does not control the real server) can independently manufacture correctly.

How the client catches the attack. When the client receives the ServerHello, it checks whether the connection is negotiating a lower version than the client itself actually offered. If it is, the client inspects those specific bytes of ServerHello.random for the downgrade-sentinel pattern. If a genuinely TLS-1.3-capable server is on the other end, it would only ever legitimately fall back to an older version when it does not actually support anything newer — in which case it would not (and structurally cannot, per the specification) embed the sentinel used for signalling "I actually support a newer version, but was tricked/forced down." If the client finds the sentinel present anyway, this proves a downgrade was engineered rather than genuine, and the client aborts the connection rather than continuing over the weakened protocol.

A second, independent safety net. Separately, the entire handshake transcript (every handshake message exchanged, including the original, unmodified ClientHello) is covered by the Finished message's authentication check at the end of the handshake. Because the attacker's tampering changes the transcript that each side computes locally, any MITM modification of the negotiated parameters causes the Finished message verification to fail, and the connection is aborted for that reason as well — giving downgrade protection two separate, overlapping layers of detection rather than relying on just one mechanism.

QUESTION 6: CRYPTOGRAPHIC CLEANING & PROTOCOL DOWNGRADES [15 MARKS]

6.1 DEPRECATION OF WEAK CIPHERS (8 MARKS)

TLS 1.3 removes weak and outdated cryptographic algorithms that were still allowed in TLS 1.2. This "cryptographic cleaning" makes the protocol simpler, safer, and reduces the chance of using insecure options.

EXAMPLES OF REMOVED (BANNED) ALGORITHMS IN TLS 1.3

Feature / Algorithm	Used in TLS 1.2	Status in TLS 1.3	Why It Was Weak / Risky
Static RSA Key Exchange (RSA key transport)	✓ Allowed	✗ Banned	No Forward Secrecy. If server private key is stolen, all past sessions can be decrypted.
DHE with <= 1024-bit parameters	✓ Allowed	✗ Banned	Small key sizes can be broken with sufficient computing power.
Export-grade cipher suites (40/56-bit)	✓ Allowed	✗ Banned	Very short keys, easy to brute force. Not secure even by old standards.
RC4 Stream Cipher	✓ Allowed	✗ Banned	Biased keystream, vulnerable to statistical attacks.
MD5 / MD2 / MDC-2 Hash Functions	✓ Allowed	✗ Banned	Broken collision resistance. MD5 collisions are practical.
Anonymous (Unauthenticated) Cipher Suites	✓ Allowed	✗ Banned	No server authentication → Vulnerable to MITM attacks.

SECURITY BENEFITS OF REMOVING OLD ALGORITHMS

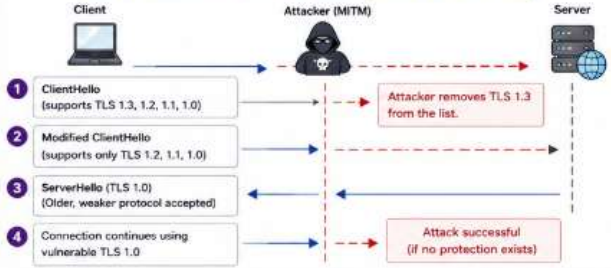
- **Stronger Security by Default:** Only strong, modern, well-analyzed algorithms remain. No weak choices to accidentally enable.
- **Eliminates Misconfiguration Risk:** Admins cannot mistakenly enable legacy ciphers for "compatibility", which attackers can exploit.
- **Reduces Attack Surface:** Fewer code paths and algorithms → smaller, simpler code base with fewer bugs.
- **Improves Performance:** Modern AEAD ciphers (e.g., AES-GCM, ChaCha20-Poly1305) are faster and safer.
- **Future-Proof:** Hard deprecations force ecosystem to move forward and avoid reliance on insecure legacy tech.

KEY TAKEAWAY:
By removing weak algorithms instead of leaving them as "optional", TLS 1.3 ensures strong security for everyone, reduces mistakes, and builds a safer internet.

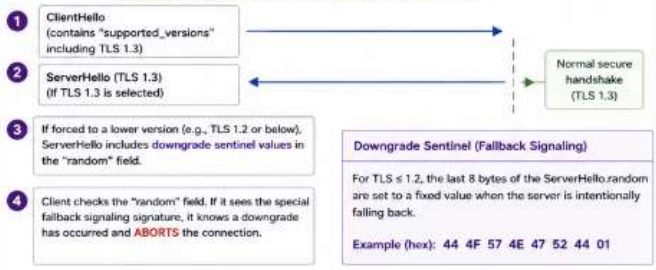
6.2 PREVENTING DOWNGRADE ATTACKS (7 MARKS)

A downgrade attack tricks the client and server into using an older, weaker TLS version (e.g., TLS 1.0 or TLS 1.1). TLS 1.3 prevents this using fallback signaling (downgrade sentinels) inside the handshake messages.

HOW A DOWNGRADE ATTACK WORKS (WITHOUT PROTECTION)



HOW TLS 1.3 PREVENTS THE DOWNGRADE ATTACK



Downgrade Sentinel (Fallback Signaling)

For TLS ≤ 1.2, the last 8 bytes of the ServerHello.random are set to a fixed value when the server is intentionally falling back.

Example (hex): 44 4F 57 4E 47 52 44 01

KEY TAKEAWAY:
TLS 1.3 uses cryptographic fallback signaling inside the handshake so that any forced downgrade is detected by the client and the connection is safely terminated.